

Daily Scholar Papers Report — 2026-09-07

2026-09-07

Daily Scholar Papers Report — 2026-09-07

[Download PDF](#)

Window covered: 2026-09-06 → 2026-09-07 (Google Scholar alerts + user-curated self-emails, last 24 h). Five alert threads arrived, none previously consumed; the standing 7-day liveness sweep returned fifteen older threads, all already accounted for by earlier runs. No backlog this morning.

Executive Summary

Nine candidates, six read, and they turn out to be arguing with each other about the same question: **what is a number allowed to claim?**

Start with the most concrete answer. **PatchGuru** observes that a regression suite is not a specification of a *patch* — it was written for the program, not for the change — so a patch that silently fails to do what its pull request says it does sails straight through. The fix is to build the missing artefact: an executable oracle, scoped to the one function the PR touched, inferred from the prose developers already wrote. It found **24 confirmed bugs against Testora's 7, at 0.62 precision against 0.32**, and maintainers fixed 11 of them. The assertion is the demonstration.

NCFuzz makes the same complaint about coverage numbers. Its authors surveyed 469 CVEs across nine network services and found **142 — more than a quarter — require a non-default configuration** to trigger. A fuzzer that pins configuration at its default therefore reports coverage over a space it has silently truncated, and cannot say so. Treating the test case as a *pair* — administrator configuration and attacker message sequence, co-evolved — buys **25.62 % average branch coverage over AFLNet and a 28.64x speed-up**, and the ablation is the interesting part: the configuration channel contributes 13.64 % on its own, the LLM-generated seeds 5.22 %, and together 25.62 %.

Then the two papers that go looking for the problem systematically, and find it. The **symbol name recovery survey** does something surveys almost never do: it re-runs the field. Fourteen artefacts were actually executable; **eleven reproduced their own published numbers within 10 %** — and then, moved onto a single neutral benchmark, **every score fell by 33.4 % to 44.4 %**. The papers were honest and the numbers were still not about the capability. And **The Data Problem** audits the substrate underneath all of it: across 93 datasets, **executable artefacts are the only type where realism and independently checked labels co-occur — 15 of 24 — while just one of 41 code-sample datasets meets the strict real-world grade**, and **49 of the 90 datasets where train/test independence applies do not address it at all**.

The remaining two supply the raw material for that critique, from opposite directions. **Athena** opens on the fact that over half of vulnerability-database entries carry missing or wrong affected-library fields, and repairs them by modelling the database as a graph rather than as loose text — where a **110M-parameter backbone already beats a 7B generative baseline**, and constraining the LLM to *select* from

a closed candidate set instead of generating freely cuts hallucinated package names **from 11.4 % to 2.9 %**. And **CVE-Smart-Contracts** curates 568 CVE records into a dataset whose defining discipline is refusal: a validated correspondence between a record and an artefact is explicitly *not* treated as confirmation that the vulnerability exists, six claims are marked refuted outright, and one 2018 mass-filing is left visible in the distribution rather than smoothed away — **431 of 491 labels are integer overflow**.

Six papers, one instinct: know what your evidence entitles you to say, and build the machinery that makes the difference legible.

Outstanding: 3 · Keep: 2 · Borderline High-Priority: 1

A note on depth and on links. All six papers were read in full text. Four carry a confirmed **CC BY 4.0** licence (NCFuzz, the symbol-name survey, Athena, CVE-Smart-Contracts); one is CC BY 4.0 on arXiv with an empty in-document copyright line (The Data Problem); one states no licence in its PDF (PatchGuru). No PDFs were mirrored into this repository on this run and no figures are reproduced — every paper below carries a canonical publisher or arXiv link instead. Two arXiv HTML sources exceeded the fetcher’s extraction cap, so **Athena’s appendices A–E were not retrievable** and a small number of its supplementary tables are cited by number only; the main body, all seven main tables and both figures were read directly.

Highlighted Papers

Title	Authors	Venue	Link
PatchGuru: Patch Oracle Inference from Natural Language Artifacts	T. Le-Cong, B. Le, T. Murray, C. Cadar, M. Pradel	Preprint, 2026 (SUTD · Melbourne · Imperial · CISPA)	arXiv:2602.05270 · author PDF · code
NCFuzz: Configuration-Guided Network Service Fuzzing	X. Bai, H. Ye, S. Zheng, F. Zhang, H. Hu, Z. Li	PACMSE Vol. 3, ISSTA, Art. ISSTA158, Oct 2026 (UC Irvine · Penn State · Dartmouth · China Telecom)	10.1145/3832249 · author PDF
A Survey of Symbol Name Recovery in Software Reverse Engineering	H. Fan, J. Wu, X. He, Y. Feng, Y. Yang, B. Xu, Q. Shi	ACM Computing Surveys, Just Accepted, Sep 2026 (Nanjing University)	10.1145/3844948 · supplementary
Athena: Vulnerability-Affected Library Identification via Knowledge Graph Completion	P. T. Duy, T. D. Yen, H. Nguyen-Huu, B. Le, Q.-T. Huynh, D. H. Vu, D. Lo, T. Le-Cong	EMNLP 2026 Main Conference	arXiv:2609.01187 · code
The Data Problem in Software Vulnerability Analysis: Artifacts, Quality, and Consumption	Y. Nong, Y. Du, T. Xu, H. Cai	Preprint (TOSEM submission), 1 Sep 2026 (Oakland · MUST · Kent State · Buffalo)	arXiv:2609.01503
Smart Contracts Claimed Vulnerable by the CVE Database, with Labels and Source Locations	M. di Angelo, G. Salzer	arXiv preprint, cs.CR, 1 Sep 2026 (TU Wien)	arXiv:2609.01186 · dataset · Zenodo

Papers

1.1 · PATCH VALIDATION · The regression suite was never a specification of the patch — infer one from the pull request prose, 24 confirmed bugs to Testora's 7 [!\[\]\(1d3a1175dd4902218e694b9c098adb83_img.jpg\) !\[\]\(e2297bc882e4a3f95bff068ab9c9101f_img.jpg\) !\[\]\(69eafe5577bb2820ded2637faee72eea_img.jpg\)](#)

PatchGuru: Patch Oracle Inference from Natural Language Artifacts

Authors: Thanh Le-Cong (Singapore University of Technology and Design), Bach Le (University of Melbourne), Toby Murray (University of Melbourne), Cristian Cadar (Imperial College London), Michael Pradel (CISPA Helmholtz Center for Information Security)

Venue: Preprint, 2026 (arXiv:2602.05270v2, cs.SE, 25 Jul 2026). No conference or journal is named in the paper.

Links. [arXiv:2602.05270](#) · [author PDF](#) · [replication package](#). **Licence: none stated** — no Creative Commons, ACM or IEEE notice appears anywhere in the PDF, so nothing is mirrored here and no figures are reproduced. **Evidence base: full text retrieved and read. Every number below is transcribed from the paper.**

The gap being targeted

Patch validation has no machine-checkable statement of what the patch was *supposed* to change, and each available substitute fails for its own reason. Regression tests are expensive to maintain, incomplete, and — the decisive point — largely not written for individual patches, so they cannot tell you whether an intended behavioural change actually happened; a patch that silently does nothing passes. Natural-language artefacts carry the intent but are informal and not executable. Code review is slow and tends toward maintainability rather than functional correctness, and LLM review tools inherit that defect, reasoning about code implicitly instead of validating runtime behaviour. The paper is careful about the nearest neighbour: Testora does observe runtime behaviour, but its oracle is *differential*, so a patch that fails to implement its intended change is structurally invisible — both versions behave identically — and intent is encoded as an LLM judgement rather than an inspectable artefact. Formal patch specifications exist but must hold over all inputs, which is why they are written by hand and never inferred.

The move: build the missing artefact, and make it small enough to infer

A **patch oracle** is described as an under-approximate yet practical form of patch specification: runtime assertions paired with concrete test inputs, embedded in a *comparison program* that holds the pre- and post-patch versions of the modified function side by side as `pre_fn` and `post_fn`. That framing buys three things at once. Scope is bounded to patch-affected behaviour, which is what makes inference tractable and validation cheap. Assertions are directly executable. And because both versions are present, the oracle can assert *cross-version* relations — the class of property a differential oracle cannot express and a single-version assertion cannot see.

The input is a PR restricted to those modifying exactly one function; the output is a triplet of oracle, comparison program and natural-language report; the standing assumption is that the pre-patch version is correct. Three assertion categories are defined — *preserved behaviour* (identical pre/post on certain inputs), *changed behaviour*, and *new behaviour*.

Four components:

Oracle inference gathers the NL artefacts (title, description, developer discussion, plus linked issues resolved by regex and fetched through GitHub’s REST API), distils away tangential content, then extracts code context — the modified function, its callees, same-file dependencies parsed out of the AST, external dependencies via imports, and, when the target is a method, the whole enclosing class so instances can be constructed. The comparison program is generated with *placeholders* for the two implementations, filled in later.

Oracle enhancement iteratively generalises assertions to broader input domains and synthesises boundary cases, while keeping existing assertions intact. This exists because a preliminary analysis found the first-pass oracles over-fitted to their own inferred inputs.

Self-review is an LLM-as-judge pass over each failing assertion, weighed against the NL artefacts, code context, diff and execution report, returning true positive or false positive with an explanation; a false positive is turned into a concrete enhancement rather than discarded.

The orchestrator substitutes implementations using the diff’s line ranges, renames functions with `pre_`/`post_` prefixes, injects dependencies, generates mocks or stubs where needed, and executes inside a Docker container. Its routing rule is the design decision worth stealing: **a violation on the *pre-patch* function is classified as an LLM reasoning error and sent to repair, not reported**. The system refuses to raise a warning it cannot attribute to the patch.

Algorithm 1 makes the loop explicit: infer, execute, then while under the iteration cap — no violation → enhance; assertion violation → self-review, and return immediately if the verdict is true positive; unexpected error → repair; re-execute; and abort with an empty report once the budget of LLM calls is exhausted. Caps: 5 LLM invocations per phase, 20 per PR, 1 h per test execution.

What the numbers say

Benchmark. 400 PRs — the 100 most recently merged from each of Keras, Marshmallow, Pandas and SciPy — after excluding doc-only and test-only PRs and keeping only single-function changes.

Inference success. Oracles were inferred for 336 of 400 PRs, a **84 % success rate**, ranging from **80 % (Keras) to 90 % (SciPy)**. The 64 failures split into unresolvable runtime errors (33) and unresolvable LLM query errors (31).

Detection. 39 warnings, **24 true positives, 15 false positives, precision 0.62**. Of the 24: 17 code bugs and 7 documentation bugs; of the 17 code bugs, **12 were still present in the latest version**, 4 had been fixed independently and 1 belonged to a since-deleted function. Maintainers confirmed all of them and **fixed 11**.

Against the state of the art. Testora produced 22 warnings and 7 true positives at **precision 0.32**; PatchGuru **detects 17 more bugs (24 vs 7)** and **22 of its 24 are unique to it**, while Testora still finds 5 that PatchGuru misses. Per-project precision runs 0.56 (Keras) to 0.73 (Pandas). A separate comparison against Codex CLI’s `/review` on GPT-5-mini is more telling than the headline: on the 24 known-buggy PRs it reached 0.52 precision, but on 40 randomly sampled PRs it reached **0.09**, and it missed 12 of the 24.

Oracle adequacy. Measured by mutation score with Mutmut over the post-patch version, the inferred oracles reach **0.70 against the developer-written regression tests’ 0.58 — 21 % higher** — with the improvement statistically significant on Keras, Pandas and SciPy (Wilcoxon signed-rank, $p < 0.05$). Combined, oracles plus regression tests reach **0.81**, with **median 1.0 on Pandas and Marshmallow**.

Cost. 8.9 minutes and roughly USD 0.07 per PR on GPT-5-mini (6.9 min on Marshmallow to 10.4 min on Pandas; ~35.7k input and ~30.2k output tokens).

Ablation. Removing oracle enhancement drops detections from 24 to 12 and mutation score from 0.70 to 0.64, while precision *rises* to 0.67 — enhancement buys recall at a small precision cost. Removing self-review is the dramatic one: **195 warnings at an estimated 0.13 precision**, and of 20 sampled warnings that self-review had filtered, **19 were confirmed false positives**.

What the authors concede

The pre-patch-correct assumption “may cause false negatives when the pre-patch itself is buggy.” Manual true/false-positive classification is human-judged, mitigated by cross-validation and maintainer confirmation. LLM non-determinism threatens reproducibility. The work is Python-only, and although the approach is described as language-agnostic in principle, supporting more languages “may require substantial engineering effort.” Single-function patches only, with the extension called non-trivial. Mutation score is a proxy that “may not perfectly correlate with real-world fault detection capability.” And, disarmingly: “due to limited funding, we evaluated only GPT-5-mini.”

Why it is worth your time

Two transferable ideas, independent of LLMs. First, **under-approximation as a design target**: the reason patch specifications were never inferred automatically is that a specification must hold universally, and dropping that requirement — scope it to the diff, accept concrete inputs — turns an intractable problem into a tractable one without turning the artefact into a guess. Second, **the pre-patch violation as a self-diagnostic**: the system has a cheap, sound signal for its own reasoning errors, because the pre-patch version is assumed correct, so any assertion it fails is the oracle’s fault rather than the code’s. Most LLM pipelines have no such signal. This one gets it for free from the comparison-program structure, and the ablation shows it is doing the heavy lifting: without the review layer that consumes it, precision collapses by a factor of five.

“Software patches are essential for system evolution but remain a major source of bugs”

1.2 · FUZZING · A quarter of network-service CVEs need a non-default configuration, so fuzz the config and the message as one pair — 25.62 % more branches, 28.64× faster 

NCFuzz: Configuration-Guided Network Service Fuzzing

Authors: Xuesong Bai (University of California, Irvine), Hengkai Ye (Pennsylvania State University), Shenghan Zheng (Dartmouth College), Fenglu Zhang (China Telecom), Hong Hu (Pennsylvania State University), Zhou Li (University of California, Irvine)

Venue: Proc. ACM Softw. Eng. 3, ISSTA, Article ISSTA158, October 2026, 24 pages. Received 2026-01-30; accepted 2026-06-25. CCS: Security protocols; Software security engineering; Domain-specific security and privacy architectures.

Links. [10.1145/3832249](https://doi.org/10.1145/3832249) · [author PDF](#). **Licence: CC BY 4.0** — “This work is licensed under a Creative Commons Attribution 4.0 International License. © 2026 Copyright held by the owner/author(s).” No PDF is mirrored on this run and no figures are reproduced. **Evidence base: full text retrieved and read.**
Every number below is transcribed from the paper.

The gap being targeted

The motivating measurement is the paper's strongest move, and it comes before any system design. Surveying nine widely deployed network services, the authors found **469 CVEs reported from 1999 to 2024, of which 142 — more than a quarter — were triggered under non-default configurations** (BIND 9 alone: 57 of 163; OpenSSH: 45 of 113). Existing network fuzzers hold configuration fixed, typically at its default, and mutate only messages — AFLNet, NSFuzz, ChatAFL and SGFuzz are named, as are OSS-Fuzz and ProFuzzBench for bootstrapping from a single configuration. So a quarter of the historical bug space is outside what those campaigns can reach, and their coverage figures do not report the omission.

The adjacent work is dismissed precisely rather than vaguely. EnvFuzz does mutate program environments including configuration, but its bugs are all triggered by *invalid* values — a corrupted option that crashes the service — which is a different target from a valid configuration an administrator would plausibly choose. ConfigFuzz encodes options as bytes alongside the input, which the paper calls syntactic integration “without modeling the runtime interaction between configurations and inputs.” CarpetFuzz and OSmart extract command-line option relations, but options arrive with the input, in the same dimension; configurations and attacker messages are orthogonal, supplied by different parties at different times. That orthogonality is the paper's whole thesis.

The move: the test case is a pair

The design principle, stated plainly, is that **ConfBugs are exposed by pairs of administrator configurations and attacker message sequences** — so NCFuzz treats each test case as a paired seed and co-evolves the two. A ConfBug is defined in prose: the administrator chooses a *valid* configuration, and the bug fires when the service receives one or a sequence of network messages. Built on AFLNet, four components:

Configuration locating and tracking is a custom LLVM pass resting on two observed patterns: services load options through a dedicated configuration-loading function, and options end up in globals or struct members that eventually flow into comparison operands. Locating finds the loading callsites by keyword matching, pulls the option-name string constant, labels the returned object as a dataflow source, and follows it forward to the backing global or struct member — the worked example resolves `stale-answer-client-timeout` to the 86th member of `struct.dns_view`. Tracking then finds accesses to those objects via structure name and member index in `GetElementPtr`, follows dataflow to a comparison instruction, and **inserts a runtime callback immediately before that comparison**, so the fuzzer learns not that an option was set but that it was actually *checked*. Inputs that trip a new option are retained as interesting and the associated option gets more energy.

LLM-aided configuration parsing first *segments* the manual into per-option blocks — feeding the whole document degrades quality — then parses each block into a structured record with Name, Type, Default, Description, Limit, Relationship, Format and a chain-of-thought Explanation.

LLM-aided seed generation produces configuration seeds conditioned on high-level feature intents (DNSSEC validation, access control, recursion and forwarding, logging and rate limiting) so they spread across subsystems, then generates compatible messages. Free-form generation fails grammar checks on binary protocols, so NCFuzz supplies an abstract message template captured from real traffic and asks the model to fill in the blanks, rendering to the wire with Scapy. Rule-based repair then fixes numeric field widths, count fields and per-record syntax; undecodable messages are dropped, and configurations are validated by actually launching the service and discarding any that terminate early.

Configuration-aware mutation and scheduling picks an option from those tracked but not present in the current seed — verbatim, “a random option $c \in C_{\text{Tracked}}$ C as a candidate for mutation” — widened by the parsed `Relationship` field to offset tracker false negatives. Mutation is type-aware: booleans flip using the software’s own literals (`yes/no`, `on/off`), numerics take the default or powers of two within the documented Limit, strings are sampled from inferred valid values or synthesised from the `Format` field, structured values (IP addresses, ACLs) are LLM-generated, and resource-pointing options draw from pre-defined constants. Algorithm 1 alternates configuration and message mutation within each round, on the reasoning that a configuration mutation has the larger effect — it can unlock entire code regions — while the feedback score supplies an adaptive ratio without explicit weighting: a configuration mutation that yields nothing scores low and energy flows back to messages.

One engineering note worth borrowing: `__AFL_INIT()` is placed **immediately before the service reads its configuration**, so the forkserver’s child re-reads configuration on every fork and no full restart is needed per configuration change.

What the numbers say

Targets and baselines. Six implementations across three protocols — BIND 9 (421k LoC) and Unbound (204k) for DNS; LightFTP (4.7k), ProFTPD (242k) and PureFTPD (32k) for FTP; OpenSSH (144k) for SSH — against AFLNet, NSFuzz and ChatAFL. **24 hours per trial, ten trials per subject**, on dual AMD EPYC 9354 with 756 GB RAM. Statistical treatment is the Vargha–Delaney $\hat{A}_{12}$ effect size; no p -values are reported.

Headline. Against AFLNet, **25.62 % average coverage improvement and 28.64× speed-up across the six services**, with the gains larger on larger codebases — **over 30 % branch coverage on Unbound and 112.7× speed-up on BIND 9**. $\hat{A}_{12}$ is 1.00 in most cells.

The honest loss. On LightFTP — 4.7k lines, the smallest subject — **NSFuzz achieves roughly twice NCFuzz’s branch coverage (387.6 vs 184)**, $\hat{A}_{12} = 0$. The paper reports it in the main table rather than burying it.

Protocol state. On OpenSSH, **117 states against AFLNet’s 59 (+98.31 %)** and **139.4 state transitions against 63.8 (+118.49 %)**. On the FTP subjects the state gains are small or slightly negative against ChatAFL.

Configuration reach. The metric that only exists because of the problem framing: NCFuzz **triggers 86.9 % of configuration options** — 43/64 on BIND 9, 187/215 on Unbound, 26/26 on ProFTPD, 85/94 on OpenSSH.

Ablation. Configuration tracking, mutation and scheduling alone give **13.64 %** average improvement; LLM-generated seeds alone give **5.22 %**; together **25.62 %**. The composition is not additive and the components are not interchangeable — on OpenSSH the configuration channel *alone* is **−21.23 %** and the seeds alone **+16.10 %**, yet combined they reach **+26.74 %**. Seed usability: **96 % of configuration seeds and 94.7 % of message seeds** were usable directly or after simple editing (6 of 150 configurations invalid; 100 of 450 messages invalid, of which the editor repaired 76).

A baseline worth noting. A naive two-step alternative — have an LLM generate configurations independently, then run stock AFLNet under each, with 240 CPU-hours distributed over ten validated configurations — is beaten by NCFuzz on every subject, and beats plain AFLNet on only two of six. Generating good configurations is not the hard part; coupling them to the message stream is.

Bugs. Five ConfBugs, **none found by any baseline fuzzer**. Three on BIND 9 match prior CVEs (CVE-2022-3736, CVE-2023-5517, CVE-2022-3924, all CVSS 7.5); a LightFTP crash on `maxusers = 0`; and an Unbound hang over DNS-over-TCP that had been independently reported before disclosure. No new CVE identifiers were assigned.

Cost. 24k LoC C/C++ plus 4.8k LoC Python; instrumentation adds under 1 MB to the binary; documentation parsing and seed generation cost **USD 0.30 per subject** and finish in 10–20 minutes.

What the authors concede

The configuration tracker has real false negatives, and the paper shows two: an option that influences another through *control flow* rather than dataflow, and a flag compared to a string and then used only as a condition, with no dataflow path to the behaviour it gates. LLM extraction errors occur. The stated external threat is documentation dependence — “documentation may be incomplete, outdated, or incorrect,” and BIND 9’s reference contained errors inconsistent with the implementation that produced invalid seeds. Exploitability may be limited if administrators rarely enable the relevant features, though the three rediscovered CVEs are all CVSS 7.5. Only configuration *files* are handled, not command-line arguments, environment variables or runtime control interfaces. And DNS being stateless, state metrics are simply uncountable for BIND 9 and Unbound.

One structural caveat, stated as fact rather than criticism: the contributions list promises “We will release the source code of NCFuzz and the new benchmark” in the future tense, and the Data Availability section states that no external datasets are used and no additional data is required. No artefact URL appears in the paper.

Why it is worth your time

The framing generalises past fuzzing. Whenever a system’s behaviour is jointly determined by two inputs supplied by two different parties on two different timescales — configuration and traffic, feature flags and requests, policy and query, model weights and prompt — testing that pins one and varies the other is measuring a slice while reporting the volume. The CVE survey is the piece to imitate: before building anything, the authors quantified how much of the historical bug space their field’s standard methodology could not reach, and got a number (142 of 469) that justifies the entire paper. The tracker design is the second transferable piece — instrumenting the *comparison* rather than the load gives a feedback signal that an option mattered, not merely that it was parsed.

“ConfBugs are exposed by pairs of administrator configurations and attacker message sequences.”

1.3 · REVERSE ENGINEERING · 11 of 14 artefacts reproduce their own published numbers, then lose 33.4–44.4 % on one neutral benchmark of 107 Coreutils binaries [!\[\]\(3211b5d1d968fc1665909b34f9f16010_img.jpg\) !\[\]\(d47ad152ec3d86a04ad64c8049e1f17f_img.jpg\) !\[\]\(6b7fbb0b7bdb78cadf73d50851a4dfb1_img.jpg\)](#)

A Survey of Symbol Name Recovery in Software Reverse Engineering

Authors: Hongcheng Fan, Jielun Wu, Xincheng He, Yang Feng, Yibiao Yang, Baowen Xu, Qingkai Shi (corresponding) — all of the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China

Venue: ACM Computing Surveys, Just Accepted. Received 11 May 2025; revised 10 August 2026; accepted 24 August 2026; online 1 September 2026. 35 pages, 132 references.

Links. [10.1145/3844948](https://doi.org/10.1145/3844948) · [supplementary material](#). **Licence:** CC BY 4.0 — “This work is licensed under a Creative Commons Attribution 4.0 International License. © 2026 Copyright held by the owner/author(s).” No PDF is mirrored on this run and no figures are reproduced. **Evidence base: full 35-page open-access text retrieved and read. Every number below is transcribed from the paper.**

Why this is not filed as “just a survey”

Because roughly half of it is an original empirical study, and that half produces the strongest negative result in this window.

The survey part is thorough on its own terms: **49 papers spanning 2000 to May 2025**, reached through a four-stage funnel (54 title matches → 993 keyword hits → 1,240 after reference and citation chasing → 49 after manual screening), of which **24 are in top-tier venues**. The field is cut along artefact type — **33 binaries, 7 bytecode, 9 source code** — and along output — **26 variable-name, 31 function-name, 8 both**. The methodological arc is laid out chronologically as rule-based → machine learning → deep learning → LLM, with the LLM tier split on the learned distribution: *discriminative* models ranking over a fixed vocabulary (BERT-family: VarBERT, DIRECT, BinAdapter, AsmDepictor) against *generative* models over an open vocabulary (general-purpose SymGen, LmPa, LLASM; domain-specific ReSym, GENNM, DOBF). For binaries alone the distribution is 2 rule-based, 5 classical ML, 12 deep learning, **16 LLM-based**.

The paper also does something small and valuable: it treats the **reverse-engineering tool as a first-class taxonomy column**, coding every approach by which decompiler it assumes — IDA Pro, Ghidra, Binary Ninja, Angr, Javap, Baksmali — because the placeholder conventions differ (`sub_N/vN` versus `FUN_N/local_N`) and the dependency is load-bearing. **34 of the 49 approaches depend on a specific tool**, and the brittleness is concrete: one system works with IDA Pro 6.95 and not 7.6 or later, because newer releases dropped Python 2.

The formal core

Unusually for a survey, the problem is given a formal characterisation — ten numbered definitions, of which the pivotal one is:

Definition 5 (Symbol Name Recovery). Symbol name recovery aims to reduce the program understanding cost: $C(P(\vec{v})) > C(P(\vec{v}'))$, while preserving program semantics equivalence: $\forall i : O(P(\vec{v}'), i) = O(P(\vec{v}'), i)$, where $\vec{v}'$ represents the recovered symbol names corresponding to $\vec{v}$.

Read it carefully and the definition says nothing about matching the *original* names — recovery is defined purely as cost reduction under semantic equivalence. Which makes the survey’s later complaint about metrics not a quibble but a consequence: every metric in use measures string agreement with the stripped-away original, and the field’s own definition does not ask for that.

The metric definitions are given too — Definition 7 for token-level precision, recall and $F1 = (2 \times P \times R) / (P + R)$; Definition 8 for symbol-level accuracy as exact match; Definition 10 for the Jaccard index over predicted and ground-truth token sets; and

Definition 9 (Character Error Rate (CER)). CER is a metric that calculates the average edit distance between the ground truth and predicted names, i.e., $CER = (1/n) \sum \text{dis}(\text{pred}_i, \text{truth}_i) / |\text{truth}_i| \dots$

Usage across the 49 papers: **Precision 24, Accuracy 20, Recall 19, F1 15, CER 5, Jaccard 2**. Notably, no BLEU, CHRF or ROUGE anywhere — the field never adopted NLG metrics.

The empirical study, which is the point

Of the 49 approaches, **23 released artefacts** (21 open-source plus two web tools) and **only 14 were actually runnable**. Three findings, in order of increasing discomfort.

Replication holds. 11 of the 14 reproduce their originally reported results within 10 %. The field is not fabricating numbers. Three diverge, and the paper is specific about why: one scores *higher* than reported (0.452 against 0.346) because the original random seed for the split was unavailable and the replication leaked; two score lower (0.712 against 0.835; 0.526 against 0.635).

Generalisation does not. Moved onto a single neutral benchmark — **GNU Coreutils v9.8, 107 binaries, compiled four ways** (GCC 15.2 and GCC 9.4, each at -O0 and -O2) — precision, recall, F1 and accuracy across all artefacts fall by **34.9–41.9 % under GCC 15.2 -O0, 33.4–40.4 % under GCC 9.4 -O0, 38.4–44.4 % under GCC 15.2 -O2, and 36.9–43.6 % under GCC 9.4 -O2**. The best absolute score on function names is **F1 0.271**. For scale: IDA Pro's own naming reaches **F1 below 0.02**, so the techniques are enormously better than the baseline and still nowhere near usable — both halves of that sentence matter.

Optimisation hurts more than compiler version. Average total F1 falls from 0.125 to 0.090 (GCC 15.2) and 0.140 to 0.106 (GCC 9.4) moving -O0 → -O2, while changing compiler version at fixed -O2 moves average F1 by only **0.016** — though one system swings **0.147**, so the aggregate hides per-technique fragility.

The authors name four candidate causes without adjudicating: domain shift, toolchain evolution, name splitting, and label noise.

The findings that generalise beyond this subfield

Six findings and seven future directions, and three of them are portable to anyone building learned tools over code:

Restrictions are structural, not incidental. Vocabulary caps (one system: 10,000 names), subword length windows (2–12 characters), AST node ceilings (under 200 nodes), token limits (under 1,024), and one approach training 16 separate models. Meanwhile **35,615 of the 120,724 symbol names in Coreutils — around 30 % — are compound words**, which a fixed vocabulary cannot emit.

The metrics are the wrong shape, and the paper proves it with three worked examples rather than argument. Against ground truth `arg_start`: the prediction `arg_begin` — a perfect synonym — and the prediction `arg_count` — semantically wrong — receive *identical* scores (P = R = F1 = 50 %, A = 0 %, CER = 56 %). Meanwhile `copy_data_range` against `copy_file_range` scores better on every metric while being the more dangerous error. And synonymy is not rare: of the Coreutils names examined, **45 (~42 %) contain `start` and 7 (~7 %) contain `begin`**.

Generalisability is not claimed, and not achieved. 39 of 49 approaches handle exactly one programming language.

What is not there

There is no Threats to Validity section and no Limitations section — the word “threat” does not appear — which for a survey that runs an original comparative study with four acknowledged confounds and one acknowledged data leak is the most notable omission in the paper. The self-limiting statements exist but

are scattered inline, and two are asserted rather than tested: that label noise is an inherent robustness requirement and so does not compromise fairness, and that Coreutils is “entirely sufficient” for cross-approach comparison. The evaluation is CPU-only, by necessity — old GPUs are hard to acquire and the released code is often incompatible with new ones, which is itself a finding about the field’s reproducibility half-life.

Why it is worth your time

The replication–generalisation split is the reusable result, and it is cleaner here than almost anywhere: the same 14 artefacts that reproduce their own numbers within 10 % lose a third to a half of their score on one unremarkable benchmark of standard Unix utilities. Nobody cheated. The numbers were simply about the datasets. That is the same claim The Data Problem (1.5) makes analytically across 93 vulnerability datasets, demonstrated empirically in a different subfield by different authors in the same week — and it is the argument for keeping a neutral held-out benchmark that no one in the field tuned against, which is cheap insurance nobody buys.

“The artifacts’ performance on Coreutils is significantly lower than that on their original dataset.”

1.4 · VULN INTELLIGENCE · Half of CVE affected-library fields are wrong; a 110M-parameter graph model beats a 7B generative baseline and cuts hallucinated package names 11.4 % → 2.9 % 🍌🍌🍌

Athena: Vulnerability-Affected Library Identification via Knowledge Graph Completion

Authors: Phong Trinh Duy, Trang Dang Yen, Hung Nguyen-Huu, Bach Le, Quyet-Thang Huynh, Dieu Hoang Vu, David Lo, Thanh Le-Cong. Institutions listed: Hanoi University of Science and Technology; The University of Sydney; Singapore Management University; The University of Melbourne; Phenikaa University; Singapore University of Technology and Design. (The HTML flattens the two-row author block, so the name-to-institution mapping is not reliably recoverable for the second row; the institution list itself is verbatim.)

Venue: EMNLP 2026 (Main Conference). arXiv:2609.01187v1, cs.SE, submitted 1 Sep 2026.

Links. [arXiv:2609.01187](https://arxiv.org/abs/2609.01187) · [replication package](#). **Licence: CC BY 4.0.** No PDF is mirrored on this run and no figures are reproduced. **Evidence base: main body, all seven main tables and both figures read in full. Appendices A–E exceeded the fetcher’s extraction cap and were not retrievable; supplementary tables 8–9 and 12–15 and Figure 3 are cited by number only and no value from them appears below.**

The gap being targeted

The opening fact is the one that matters for anyone consuming vulnerability feeds: **more than half of vulnerability-database entries carry missing or incorrect affected-library information.** Prior work splits three ways and all of it is text-only. Extreme multi-label approaches (Chronos and predecessors) treat each library as an opaque label and each CVE as an isolated document, discarding the semantics carried by library names and descriptions. Semantic matching (VulLibMiner) scores candidates by embedding similarity between CVE and library text. Generative approaches (VulLibGen, LibAlarm) fine-tune LLMs to emit package names and hallucinate libraries that do not exist — a rate the paper measures at **11.4 %**. The unifying diagnosis: existing approaches overlook the relational structure sitting in plain sight inside the databases — which CWE class the vulnerability belongs to, which CPE product it

grounds out in, which ecosystem the library lives in, which libraries have historically co-occurred — because text-based methods cannot see it.

The move: model the database as the graph it already is

Modelling. A security knowledge graph over five entity types — CVE, CWE, CPE product, library, ecosystem — and five directed relations: `affect_library` (the prediction target, from expert-verified annotations), `affect_product` and `type_of_vulnerability` (from NVD metadata), and `categorise_library` and `tag_library` (from Maven Central metadata, the latter capturing ecosystem, vendor, domain and core job). Every entity carries an identifier plus a textual description. The resulting graph: **7,680 entities and 24,714 triples across 5 relation types.**

Completion. Identification is reformulated as knowledge-graph completion — predict the missing tail of `(CVE, affect_library, ?)` by link prediction. The backbone is a text-based contrastive bi-encoder (SimKGC, or the relation-aware variant RAA-KGC), which matters because being text-based makes it **inductive**: it can rank a library it never saw in training, from its identifier and description alone.

Re-ranking. The top-30 candidates go to an LLM which **selects from that closed set rather than generating freely**, prompted with the candidate list, neighbouring graph facts, and KGC embeddings projected into the model's representation space through a learned adapter. Two adaptations for the multi-label setting: multi-instance instruction tuning (several training instances per query, one per ground-truth label) and promote-then-fill inference (selected candidates go to the top in order, remaining slots filled from the backbone ranking).

The evaluation metrics are defined explicitly, with F1@k carrying the paper's only equation number:

$$P@k = |\text{pred}_k(v) \cap \text{affected}(v)| / k, R@k = |\text{pred}_k(v) \cap \text{affected}(v)| / |\text{affected}(v)|,$$

$$F1@k = (2 \cdot P@k \cdot R@k) / (P@k + R@k) \dots (1)$$

with Avg. F1 the arithmetic mean of F1@1, F1@2 and F1@3.

What the numbers say

On VulLib — 2,853 manually verified Java CVEs, split 1,668 / 620 / 565 — Athena reaches **Avg. F1 0.602** against VulLibGen 0.457, LibAlarm 0.442, Chronos 0.435 and VulLibMiner 0.381: **+32 % over the best baseline**, McNemar $p = 2e-16$.

Three results are more interesting than the headline.

Scale is not the lever. The KGC backbone alone, at **110M parameters**, reaches Avg. F1 0.470 — already above VulLibGen's 0.457 at **7B parameters**, a 64× smaller model. Re-ranking then adds 28 % on top.

The inductive claim holds. On top-3 recovery split by whether the library was seen in training, VulLibGen drops from 48.17 % (seen) to 32.31 % (unseen) — **15.86 points**. Athena drops from 63.08 % to 60.71 % — **2.37 points**.

Closed-set selection is doing real work. Only **2.9 % of Athena's top-3 predictions** contain identifiers absent from the package registry, against VulLibGen's **11.4 %** — an approximately four-fold reduction, achieved architecturally rather than by post-filtering.

Two ablations are worth recording. Relation ablation ranks the graph's ingredients by contribution: removing library tags costs 8.5 % of Avg. F1, CPE 2.7 %, CWE 2.2 %, categories 0.5 % — the free-text tags matter far more than the formal taxonomies. And robustness to an incomplete graph is good: removing 10 % of edges costs 2.0 %, removing 30 % costs 5.3 %. Fine-tuned open-weight re-rankers (0.602) beat zero-shot proprietary models (0.548–0.561), which is a result about adapters and graph-embedding injection rather than about model quality.

The failure analysis is unusually candid: for **53 test CVEs (9.4 %)** the backbone retrieves no ground-truth library in its top 30 at all, and the modes are named — **25 (47 %) new libraries**, 18 intra-family distractors, 6 semantic mismatches, 4 cross-ecosystem redistributions. Thirteen of the 53 are Jenkins `workflow-*` plugins.

What the authors concede

Re-rankers only up to 31B parameters, with larger models beyond available compute. Evaluation is Java-only because VulLib is the only benchmark carrying the library metadata that text-based KGC requires — the multi-ecosystem datasets released with VulLibGen lack those fields — so transfer to Python, JavaScript and Go remains open. And a careful temporal-leakage admission: the split reflects identifier assignment rather than disclosure time, and the CWE/CPE/library metadata come from current database versions, so fields may postdate a test CVE. “Our split therefore prevents whole-CVE overlap but cannot guarantee that every field predated the cutoff.” The ethics section notes dual use and warns that misses fall disproportionately on newly introduced libraries, so the tool should augment rather than replace analyst sign-off — a caveat the failure analysis directly supports, since new libraries are 47 % of the unrecoverable cases.

Why it is worth your time

The transferable idea is **structural hallucination control**. Everyone knows that constraining a generative model reduces hallucination; the number here quantifies what one particular constraint buys — retrieve-then-select over a closed candidate set, 11.4 % → 2.9 % — without a downstream filter and without giving up the LLM's judgement, because the model still ranks. The second is the relation ablation as a design tool: it says that in this domain the informal, human-written tags carry more signal than the curated CWE and CPE taxonomies, which is a mildly uncomfortable finding for anyone investing in taxonomy hygiene, and exactly the sort of thing you only learn by ablating relation types one at a time.

“existing approaches overlook the rich relational structure present in vulnerability databases”

1.5 · DATASETS · 1,522 papers screened, 111 deep-coded: only 1 of 41 code-sample datasets is fully real-world, and 49 of 90 never address train/test leakage [👉🤔👈](#)

The Data Problem in Software Vulnerability Analysis: Artifacts, Quality, and Consumption

Authors: Yu Nong (Oakland University), Yao Du (Macau University of Science and Technology), Tianxiang Xu (Kent State University), Haipeng Cai (University at Buffalo, SUNY)

Venue: Preprint targeted at ACM TOSEM; arXiv:2609.01503v1, cs.SE, submitted 1 Sep 2026. CCS: Software and application security; Software verification and validation; Machine learning.

Links. [arXiv:2609.01503](https://arxiv.org/abs/2609.01503). **Licence:** CC BY 4.0 on arXiv, though the in-document copyright line reads “© none” and the PDF carries only “Manuscript submitted to ACM”. Given that inconsistency, nothing is

mirrored here. **Evidence base: full text read across HTML and PDF sources; §1–§5.4 transcribed directly, later sections and appendices verified against the rendered HTML.**

The gap being targeted

Data quality in vulnerability research has been studied in pieces — accuracy and consistency audits of five widely used datasets, label-noise studies, work on train/test duplication, on undecidable labels, on benchmarks that reward the wrong capability. What has not existed, in the authors’ words, is one that “spans the full artifact range from metadata to execution traces, or that reports, per attribute, how much it has been studied separately from how well datasets achieve it.” That last clause is the paper’s real contribution: it separates *attention* from *achievement*, and the gap between the two is where the findings live.

The move: a taxonomy with a rule against itself

Three aspects, no attribute in more than one. **Artifact** covers type — code samples, metadata, patches and fixes, tests/PoCs/exploits, reasoning, traces and logs — plus granularity and provenance. **Quality** covers realism (synthetic / injected / mined / real-world), label evidence (verified / patch-derived / tool-derived / description-derived / unlabeled), scale, diversity, leakage and contamination, and availability. **Consumption** covers the consuming task and the consuming technique.

Two things make it more than a list. The vocabularies distinguish *not addressed* from *n/a* — the question applied and went unanswered, versus the question did not apply — which is what makes counting silence meaningful. And the coding protocol carries an explicit anti-hallucination rule: **no affirmative rubric-graded value is recorded unless a verbatim span from the paper supports it**, with absence values checked by targeted full-text scans. 111 anchor papers, **729 evidence spans**, every one of the 111 carrying at least one.

The corpus: 2,112 de-duplicated candidates from arXiv, DBLP and supplemental search, **1,522 included** after screening, **111 deep-coded** by four admission criteria designed so that no artefact type is crowded out by the sheer volume of code-sample work. Of the 111, 93 contribute a dataset and 18 study data they did not build.

What the numbers say

The evidence ladder. Executable artefacts — tests, PoCs, exploits — are the only major type where realism and independently checked labels co-occur: **15 of 24** are both real-world and independently label-checked (16 of 24 real-world, 23 of 24 checked). Code samples, **the largest category in both the full corpus (753 papers) and the anchor set (55)**, are the least realistic: **20 of 41 draw vulnerabilities from authentic projects or CVEs, only 3 keep the sample at the unit the code is actually deployed in, only 2 do both — and just 1 of 41 meets the strict full-context real-world grade.**

Leakage. Of the 90 datasets where train/test independence applies, **49 (55 %) do not address it at all**; a stricter count puts 54 of 90 with no effective guard. Among the 41 that do: deduplication 15, chronological 13, contamination-controlled 5, random 5, project-disjoint 3.

Availability. 62 of 93 public, **25 of 93 make no statement at all**, 5 explicitly withheld.

Attention versus achievement. Papers producing an actual *finding* about each attribute: label evidence 14, leakage 8, realism 8, diversity 7, availability 6 — and **scale, 0. Not one paper in the anchor set produces a finding about scale as such**, despite scale being the attribute most prominently advertised.

Language siloing. Of 93 datasets, 33 name no language; of the 58 that do, **45 are confined to a single language** and only 7 span three or more. C/C++ leads at 27, and **21 name C/C++ and nothing else**.

A Simpson's paradox. The verified-label share appears to improve over time — 50 % through 2023, 56 % in 2024–25, 67 % in 2026 — but held *within* artefact type it vanishes: code samples go from 3 of 8 to 2 of 7. The aggregate rises only because cohort composition shifts, as executable artefacts grow from 25 % to 37 % of new datasets. The field is not getting more careful; it is producing more of the type that is careful by construction.

Consumption asymmetry. Of 41 code-sample datasets, 25 are consumed by a learned model. Of 24 executable datasets, **none is used for training** — 17 serve as benchmarks and 16 agentically. The most trustworthy artefacts are used only to grade, never to teach.

What the authors concede

§7 is candid to the point of undercutting its own headline. “The six quality attributes and their values are ours, not the papers’, so every graded value is an interpretation,” and the mitigations “yield auditability rather than reliability” — a dissenting reader can see what each judgement rested on, but inter-coder reproducibility is not established. Cohen’s κ on the flagged disagreements is **0.47, 0.65 and 0.65**, and the paper is explicit that the 96 % mean pairwise agreement is agreement on flagged questions, not a reliability coefficient for applying the taxonomy from scratch. The 111-paper sample is purposive, “the right sample for characterizing the field’s anchor artifacts and the wrong one for estimating a population mean, and the direction of the resulting bias is not identifiable.” Re-screening all 808 excluded records returned 217. The corpus-wide auto-tagger agrees with manual typing only **70.7 %** of the time, “so no compositional claim in this study rests on it.” And on the realism finding itself: “The figure above is therefore not a free-standing discovery: it partly restates the coding rule.” Verification found one systematic pipeline fault and one record coded from a sentence about a different dataset, with **94 of 99 verdicts upheld**. The 668 LLM-assisted screening decisions are “inspectable per record but not re-executable” — no single prompt, model version or temperature can be reported.

Artifact availability: Appendix C lists the released materials — harvesting scripts, screening rules, the full candidate table with dispositions, the codebook, the 111 coded records with their 729 spans, verification worksheets and analysis scripts — but states no URL.

Why it is worth your time

Not for a verdict on any dataset, which it deliberately does not give, but for the **datasheet in Table 10** — the minimum a dataset release should state, per attribute, and what a consumer cannot assess without it. That is directly actionable, and cheap to adopt. The second reason is the *not addressed / n/a* distinction, which is a general-purpose instrument: most survey methodologies cannot count silence because they cannot tell an inapplicable question from an unanswered one, and the 49-of-90 leakage figure only exists because this one can.

“It motivates a shift from data that asserts vulnerabilities to data that demonstrates them.”

1.6 · SMART CONTRACTS · 568 CVE records curated to 491 with function-level locations, 6 refuted outright, and one 2018 mass-filing left visible: 431 of 491 labels are integer overflow 🙌🤔👉

Smart Contracts Claimed Vulnerable by the CVE Database, with Labels and Source Locations

Authors: Monika di Angelo, Gernot Salzer — Institute of Computer Engineering / Institute of Logic and Computation, Informatics, TU Wien, Vienna, Austria (both also listed with the Division of Theoretical Computer Science, KTH Royal Institute of Technology, Stockholm)

Venue: arXiv:2609.01186v1, cs.CR, submitted 1 Sep 2026. Structured as a data descriptor (Background & Summary / Methods / Data Records / Technical Validation / Usage Notes).

Links. [arXiv:2609.01186](https://arxiv.org/abs/2609.01186) · [dataset repository](#) · [Zenodo archive](#). **Licence:** CC BY 4.0 for the paper; the dataset's own metadata, schemas, labels and review decisions are CC BY 4.0 and its code MIT. No PDF is mirrored on this run and no figures are reproduced. **Evidence base: full text read. Every number below is transcribed from the paper.**

The stated claim

Evaluating smart-contract analysis needs ground truth with known types *and* locations, and the two existing CVE-derived corpora are thin: one associates 487 contracts with CVE records, the other is a 200-contract subset selected for program repair. Neither runs through 2026, archives the external evidence needed to interpret the claims, assigns taxonomy labels, or localises vulnerabilities to functions. **CVE-Smart-Contracts** is built to fill that: 568 curated CVE records up to July 2026 referring to Ethereum smart contracts, with source and runtime bytecode, labels under three taxonomies, and function-level locations — **automated to the point of leaving 15 % to manual analysis**, with all external inputs retained so rerunning the pipelines reproduces the outputs.

Five phases. **Acquisition** starts from a pinned CVE v5 commit and selects candidates by text and reference cues, checked to subsume both prior datasets. **Correspondence validation** encodes two separate claims per record — that the CVE description refers to the downloaded source, and that the named contract was deployed at the indicated address — as 20 Boolean flags per record; automatic rules settle 450 matches and 39 non-matches, 53 go to reviewed decision, final tally **497 matches and 45 non-matches**. **Labelling** applies 13 declarative rules across six evidence tiers — CVE descriptions, titles, curated evidence, referenced documents, derived or OCR'd text, and finally source artefacts — with the strongest tier containing a match deciding, and multiple matches at that tier treated as *ambiguous rather than ranked*. Three taxonomies are assigned independently: a 2026 hierarchical smart-contract taxonomy, the SWC registry, and CWE. **Localisation** compiles each contract at its metadata-indicated Solidity version, normalises legacy and compact ASTs into an inventory of source units, functions, modifiers and call edges, then traverses call and modifier edges *backwards* to identify externally reachable entry points. **Catalog construction** is fully generated, never hand-edited, and the rebuild verification fails if any tracked output differs from the committed release.

The discipline that makes it interesting

The paper is built around refusals, and states them as design rules rather than caveats. A validated correspondence is not evidence the code is vulnerable: “A correspondence match must not be interpreted as an independent vulnerability finding.” A non-match rejects the indexed correspondence, not the upstream CVE claim. Localisation confidence “express[es] evidential support for a location, not the probability that the vulnerability claim is true.” Upstream CNA weakness assertions are preserved

and compared rather than silently substituted. And where review found convincing evidence a claim was a false positive — in one case the supposedly vulnerable function was commented out — the record is marked **refuted**: six such records are excluded from the catalogue but retained in the index, so the exclusion is visible rather than invisible.

The distribution is left equally visible. **431 of 491 labels are integer overflow**, an artefact of a 2018 mass-filing of 401 reports by a single organisation, after which smart-contract CVE submissions never exceeded 11 in any subsequent year. The authors report the skew as a known limitation — toward Ethereum, toward Solidity 0.4.x, toward integer bugs — rather than rebalancing it away. Statement-level precision is honestly bounded too: **only three CVE records contain sufficient data to identify the vulnerable line**, so that field is not exported at all.

Final ledger: 568 records → 542 contracts + 26 projects → 497 matches → **491 in the released catalogue**, plus 45 without validated artefacts and 6 refuted. Automatic labelling produced 524 suggestions and 44 abstentions; the release contains 512 rule-based classifications, 55 manual, and one retained as not classifiable. Automatic localisation covered 415 records, manual decisions a further 76.

Why it is flagged Borderline rather than higher

Its value is real but narrow: a benchmark for Ethereum smart-contract analysis, dominated by one weakness class from one filing episode, and — by construction — carrying no independent verification that any of the 491 claims is true. That is the paper's own position, stated repeatedly and clearly, and it is why the dataset is more useful for *localisation* evaluation than for detection ground truth. If you work on smart contracts it is the best available artefact of its kind; if you do not, read the two pages on refutation and evidence tiering and move on.

“The dataset does not validate the original vulnerability claims”

Cross-Paper Synthesis

Six papers, one distinction: what a number asserts and what it demonstrates

Nothing coordinated this. Two arrived through followed-researcher alerts, two through Scholar's recommendation engine, and they were posted within a week of each other by groups in Singapore, Nanjing, Vienna, Irvine and Buffalo who are not working on the same problem. But run them together and each one turns out to be locating a specific place where a reported number has quietly stopped being about the thing it names.

PatchGuru finds it in the regression suite. A green build asserts that the program still works; it does not demonstrate that the patch did what its author said. The gap is not an oversight — the tests predate the patch and were written for a different purpose — and closing it requires manufacturing an artefact that never existed, scoped narrowly enough to be inferable. The ablation quantifies exactly how much of the resulting system is the *refusal* machinery rather than the inference machinery: strip self-review and warnings go from 39 to 195 while precision falls from 0.62 to roughly 0.13.

NCFuzz finds it in coverage. Branch coverage under a fixed default configuration asserts progress through the reachable program; it does not demonstrate progress through the program an administrator will actually run — and 142 of 469 historical CVEs live in the part that was never reachable. Note that the

fuzzers being criticised were not doing anything wrong by their own lights. Their number was correct. It was answering a smaller question than the one readers took it to answer.

The symbol name recovery survey finds it in reported accuracy, and — uniquely in this window — *measures the gap directly*. Eleven of fourteen artefacts reproduce their published scores within 10 %. Move them one benchmark sideways and everything drops by a third to a half. The published numbers were honest, reproducible, and about the dataset.

The Data Problem finds it in the datasets underneath all of the above, and supplies the vocabulary the other five are groping for: the difference between an artefact that *asserts* a vulnerability (a label, a CVE description, a CWE code) and one that *demonstrates* it (a PoC that runs, a test that fails, a trace). Its central table ranks the six artefact types by exactly that criterion, and its central finding is that the demonstrative end of the ladder is where realism and independent label checks finally co-occur — 15 of 24 for executable artefacts, 1 of 41 for code samples.

Athena and **CVE-Smart-Contracts** work the same seam from the supply side. Athena's premise — over half of affected-library fields wrong — and the smart-contract dataset's six refuted records are the same observation at different resolutions: the metadata that everyone treats as ground truth is an assertion by a reporter, unverified, sometimes about a function that was commented out. One paper repairs the assertions at scale; the other declines to launder them, and marks the ones it cannot stand behind.

The convergent design move: build the refusal in

The sharper pattern is not the diagnosis but the response, because five of the six systems handle it the same way — by making the boundary of a claim *structural* rather than advisory.

PatchGuru routes a pre-patch assertion violation to repair instead of to the bug report, because that violation cannot be the patch's fault under its own stated assumption. Athena has the LLM *select* from thirty retrieved candidates rather than generate, and the hallucination rate falls from 11.4 % to 2.9 % — no filter, no post-processing, just a smaller space to be wrong in. CVE-Smart-Contracts refuses to let a validated correspondence be read as a vulnerability finding, refuses to export statement-level locations when only three records support them, and marks the six claims it can disprove. The Data Problem refuses to record any affirmative graded value without a verbatim supporting span, and distinguishes *not addressed* from *n/a* so that silence can be counted. NCFuzz reports the one subject where a baseline beats it by 2× in the main table.

That is a genuine methodological convergence, and it is the opposite of the usual instinct. The easy way to raise a headline number is to let the system answer whenever it can; these five all spend capability to buy the right to be believed, and in at least two cases the ablation shows the refusal machinery contributing more than the generative machinery it constrains.

Where the tension actually is

It would be tidy to say the field is converging on demonstrative evidence. It is not, and two of these papers say why.

The Data Problem's consumption asymmetry is the hard version: of 24 executable datasets, **none is used for training** — they serve as benchmarks and as agent environments only. The most trustworthy artefact type is used exclusively to grade and never to teach, while the least realistic type, code samples, is what 25 of 41 learned models are trained on. Demonstrative data is expensive (78.9 % of CVEs have

no public PoC; 70.2 % of CVE patches ship no test file), and expensive data gets used sparingly, which means at evaluation time. So the ladder does not describe a migration. It describes a division of labour in which the training signal stays cheap and assertive, and only the scoreboard gets to be demonstrative.

The symbol-name survey shows what that division costs when the scoreboard is also drawn from the training distribution. And its Definition 5 exposes a second, quieter version of the same problem: the field's own formal definition asks only for reduced comprehension cost under semantic equivalence, while every metric in use measures string overlap with the original names — so `arg_begin` and `arg_count` score identically against `arg_start`. The metric is not measuring the definition. That is the assert/demonstrate gap relocated into the evaluation criterion itself, where it is much harder to see.

A methodological note worth stealing

Three of these papers open by quantifying the size of their own gap before proposing anything, and in each case the number is what makes the paper unarguable: **142 of 469** CVEs unreachable at default configuration; **49 of 90** datasets with no leakage guard; **33.4–44.4 %** score drop on a neutral benchmark. None of those required the system being proposed. All three could have been produced by a graduate student with a spreadsheet and two weeks, and each one is more load-bearing than the technique it motivates. If you are choosing a project this quarter, the cheapest strong move available is to measure how much of your field's problem space its standard methodology cannot reach — and then say so with a fraction.

Writing & Rationale Insights

Report the loss in the main table. NCFuzz's Table 3 contains a cell where a baseline achieves twice its coverage, with the effect size printed as 0. It would have been trivial to relegate LightFTP to a footnote — it is 4.7k lines, the smallest subject, and arguably outside the paper's stated target of large services. Keeping it visible costs one bad-looking row and buys the reader's trust in the other twenty-three cells, which is a good trade at any price.

Ablate the guardrails, not just the features. Most ablation sections remove the parts that generate capability. PatchGuru's removes the part that suppresses output, and the result — 195 warnings at 0.13 precision instead of 39 at 0.62 — is the single most informative number in the paper, because it converts a design choice everyone would have waved through into a measured five-fold effect. If your system has a filtering, review or abstention stage, that stage is a first-class contribution and should be ablated like one.

Quantify the gap before proposing the fix. See the synthesis above. A survey of 469 CVEs is not a contribution in the usual sense, and it does more work than the fuzzer.

Distinguish “not addressed” from “not applicable.” The Data Problem's vocabularies keep those apart, which is the only reason it can report that 49 of 90 datasets are silent on leakage. Most coding schemes collapse the two into a single blank cell and thereby destroy exactly the finding they were built to surface. This is a general instrument: if you are building any rubric, decide early whether an empty cell means the question failed or the question did not apply, and encode the difference.

Say what your confidence score is a score of. CVE-Smart-Contracts states that its localisation confidence expresses evidential support for a location, not the probability the vulnerability claim is true.

That sentence prevents a specific, likely, and otherwise unrecoverable misreading of the dataset by every downstream consumer. One sentence.

Run your own field before summarising it. The symbol-name survey's taxonomy would have been a respectable CSUR paper on its own. What makes it worth reading twice is that the authors obtained the 23 released artefacts, discovered only 14 would run, replicated those, and then put them all on one neutral benchmark. The replication result is reassuring; the generalisation result is not; and neither could have been written from the papers alone.